

第2章: TypeScript の基礎

この章では、TypeScript（タイプスクリプト：JavaScript に「型」の仕組みを追加したプログラミング言語。読みは「タイプスクリプト」）の基本を学びます。TypeScript は JavaScript（ジャバスクリプト：ブラウザで動く代表的なプログラミング言語）に「型（Type：データの種類のこと。例えば「文字」「数値」「真偽値」など）」という仕組みを追加した言語です。「型って何？」という方も安心してください。身近な例えを交えながら、一つずつ丁寧に解説します。

この章で学ぶこと

- **型（Type：タイプ）とは何か** — データの種類を指定する仕組み。「この箱には数字だけ入れていいよ」というラベルのようなもの
- **基本的な型** — `string`（ストリング：文字列を表す型）、`number`（ナンバー：数値を表す型）、`boolean`（ブーリアン：true/false の真偽値を表す型）など
- **インターフェース / 型エイリアス** — 自分だけのオリジナルの型を定義する方法。書籍管理アプリでは「Book型」を作ります
- **ジェネリクス（Generics：ジェネリクス。「総称型」と訳される。型をパラメータ（後で決める変数のようなもの）として受け取る仕組み）** — 型をパラメータ（引数）として受け取る柔軟な仕組み
- **よくあるエラーと対処法** — 初心者がつまづきやすいTypeScriptのエラーメッセージと、その読み方

なぜTypeScriptを学ぶの？ JavaScriptだけでもアプリは作れますが、TypeScriptを使うと「コードを書いている途中で間違いに気づける」「エディタ（コードを書くソフト）が賢く補完してくれる」というメリットがあります。最初は少し面倒に感じるかもしれませんが、慣れると「TypeScriptなしでは開発できない！」と思うようになります。

大事なポイント（先に伝えておくと安心）：

- **型は実行時には消える** — TypeScript の型情報は「人間とTypeScript コンパイラだけが見えるメモ」。プログラムが実際に動くときには影響しません（これを「型消去」と呼びます）。
- **`const` をデフォルトにする** — 値を入れ替える必要がないなら必ず `const`（再代入禁止）。「ここは入れ替わらない」とコードを読む人に伝えられて、バグも減ります。
- **`null` と `undefined`** — どちらも「値がない」を表しますが、`undefined`（アンディファインド：未定義）は「まだ何も入れていない」、`null`（ヌル：明示的な空）は「意図的に空にしている」を表します。
- **セミコロン `;` は省略可能** — 文末の `;` は本当は無くても動きますが、本書では分かりやすさのため付けます。

目次

0. 前提知識: JavaScriptの基礎の基礎
1. TypeScript とは
2. 基本的な型
3. 型注釈と型推論

4. インターフェースと型エイリアス
5. ジェネリクス
6. ユニオン型とリテラル型
7. TypeScript の設定 (tsconfig.json)
8. よくあるエラーと対処法

0. 前提知識: JavaScriptの基礎の基礎

TypeScript は JavaScript に「型」を追加した言語なので、JavaScript の文法をひと通り知っておくと話が早く進みます。ここでは TypeScript を読むうえで最低限必要な JavaScript の文法だけ、ぎゅっと圧縮して解説します。「もう知ってる！」という方は読み飛ばして 1 章へ。

動かしながら読みたい方へ: 以下の例はどれも、ターミナルで `node` と入力して Enter を押すと開く対話モード (REPL) に貼り付けて実行できます。終了は `Ctrl+C` を2回押します。または、ブラウザの開発者ツール → **Console** (F12 で開ける) にそのまま貼り付けてもOKです。

0.1 コメント

コメントは「コンピュータには無視されるが、人間が読むためのメモ」です。

```
// この行はコメント。実行されない。
let x = 1; // 行末にも書ける

/*
 * 複数行コメント。
 * /* と */ で囲む
 */

// 行頭の // からその行の終わりまでが「単一
// let = 後から値を変えられる変数の宣言
// ↑ 空行も自由に入れられる (コードの
// /* で始めると複数行コメントの開始
// * は飾り (書式の慣習)。意味はない。
// 文章中の /* や */ も「ただの文字」
// */ で複数行コメントの終了。
```

0.2 変数の宣言: `const` / `let`

値に名前を付けて保存する仕組みです。本書では原則 `const` (コンスト: constant の略。再代入できない変数) を使い、必要なときだけ `let` (レット: 再代入できる変数) を使います。古い書き方の `var` (バー: variable の略。古い変数宣言) はバグの元なので使いません。

なぜ `const` をデフォルトにするのか? 変数を見ただけで「これは中身が変わらない」と分かったら、コードを読む人の負担が減ります。「もしかして他で書き換えてる？」と疑う必要がないからです。React/Next.js のコードではほぼ全ての変数を `const` で宣言します。

```

const name = "太郎"; // const = 一度入れたら変えられない // const は「定数」宣言。na
// name = "次郎"; // ❌ エラー: Assignment to constant variable. // ↑ コメントアウトして

let count = 0; // let = 後で書き換え可能 // let は「後で値を変えら
count = count + 1; // ✅ OK // count に「現在の
console.log(count); // console.log は「

// ▼ 実行結果
// 1

```

console.log() とは?: ターミナル (Node.js の場合) またはブラウザのコンソール (ブラウザの場合) に値を表示する関数です。プログラムの動作確認に多用します。

0.3 基本のデータ型

データ型	例	説明
文字列 (string)	"hello" '太郎' `テンプレート \${x}`	クォートで囲んだ文字
数値 (number)	42 3.14 -0.5	整数も小数も同じ型
真偽値 (boolean)	true false	ハイorロー
null / undefined	null undefined	値が「ない」ことを表す (null=明示的、undefined=未定義)
配列 (array)	[1, 2, 3]	値の並び
オブジェクト (object)	{ name: "太郎", age: 20 }	キー: 値の組

```

const text = "hello"; // 文字列リテラル "hello" ("..." または '...' を用いて)
const num = 42; // 数値リテラル 42 (クォートで囲わない数字) を用いて
const isOk = true; // 真偽値 true (true か false の2つしかない)
const list = [1, 2, 3]; // 配列リテラル。[ ] の中に値を , で並べる。[0] は 1
const user = { name: "太郎", age: 20 }; // オブジェクトリテラル。{ キー: 値 } の組を ,

console.log(text, num, isOk, list, user); // console.log は引数を " , " 区切りでまとめて表示

// ▼ 実行結果
// hello 42 true [ 1, 2, 3 ] { name: '太郎', age: 20 }

```

0.4 演算子（足し算・比較）

```
console.log(1 + 2);           // 3（足し算）           // + は加算演算子。数値同士が
console.log(10 - 3);          // 7                   // - は減算演算子。10 から
console.log(4 * 5);           // 20                   // * は乗算演算子（× の代
console.log(10 / 3);          // 3.3333333333333335     // / は除算演算子（÷ の代
console.log(10 % 3);          // 1（余り）             // % は剰余演算子。10 ÷ 3
console.log("ab" + "cd");     // "abcd"（文字列の連結） // + の左右が文字列だと、足

console.log(1 === 1);         // true（等しい。型もチェック） // === は「型も値も完全に同じ
console.log(1 === "1");       // false（型が違っていると false） // 数値の 1 と文字列の "1"
console.log(1 !== 2);         // true（異なる）           // !== は「型または値が異な
console.log(3 > 2);           // true                   // > は「左が右より大きい
console.log(true && false);    // false（両方trueならtrue = AND） // && は AND 演算子。両辺
console.log(true || false);   // true（片方trueならtrue = OR）  // || は OR 演算子。どち
console.log(!true);           // false（反転）           // ! は否定演算子。true →
```

`==` ではなく `===` を使う: 1個少ない `==` は型変換しながら比較するため、`1 == "1"` が `true` になります。バグの温床なので、本書では一貫して `===` を使います。

0.5 文字列とテンプレートリテラル

```
const name = "太郎";           // 文字列リテラルを変数 name に代入。

// 連結（古い書き方）
const msg1 = "こんにちは、" + name + "さん!";           // + 演算子で「文字列+変数+文字列」をつなげ

// テンプレートリテラル（新しい書き方。バッククォート ` で囲む）
const msg2 = `こんにちは、${name}さん!`;           // `...` で囲んだ中に ${変数} を書くと、その

console.log(msg1);               // 1行目の文字列を表示。
console.log(msg2);               // 2行目の文字列を表示。

// ▼ 実行結果
// こんにちは、 太郎さん！
// こんにちは、 太郎さん！
```

`${ ... }` の中には変数や式（しき：値を計算する記述。`1 + 2` や `name.length` など、評価すると1つの値になるもの）を入れられます。テンプレートリテラル（template literal：バッククォートで囲んだ文字列）は改行も含められて便利です。

0.6 if文（条件分岐）

```
const score = 75; // 変数 score に数値 75 を入れる。

if (score >= 80) { // if は「もし～なら」の意味。( ) 中の条件式が
  console.log("優"); // score が 80 以上のとき表示。
} else if (score >= 60) { // else if = 「上の条件に当てはまらず、こちらの
  console.log("可"); // 60 以上 80 未満なら表示。
} else { // else = 「どの条件にも当てはまらないなら」。
  console.log("不可"); // 60 未満なら表示。
} // if 文の終わり。

// ▼ 実行結果
// 可
```

`{ ... }` で囲んだ範囲を**ブロック**（block：複数の文をひとまとめにした区切り。中括弧で囲む）と呼びます。条件に当てはまったブロックだけ実行されます。

「文（statement）」と「式（expression）」の違い: `if (...) { ... }` のように「処理を1つ実行する命令」が文。`1 + 2` や `score >= 80` のように「評価すると1つの値になる」のが式。`const x = 1 + 2;` は「`x` に `1+2`（式）を代入する文」のように、文の中に式が入ります。

0.7 for ループ・配列のループ

```
// 0, 1, 2 と3回繰り返す
for (let i = 0; i < 3; i++) { // for ループ。( 初期化; 継続条件; 各回の最後の
  console.log(`i = ${i}`); // ループの中身。テンプレートリテラルで現在の i
} // for ブロックの終わり。

// ▼ 実行結果
// i = 0
// i = 1
// i = 2

// 配列を1つずつ取り出す（モダンな書き方）
const fruits = ["apple", "banana", "cherry"]; // 文字列の配列を作る。
for (const fruit of fruits) { // for...of ループ。fruits の中身を 1つずつ fr
  console.log(fruit); // 取り出した値を表示。
} // for ブロックの終わり。

// ▼ 実行結果
// apple
// banana
// cherry
```

0.8 関数の書き方

関数は「処理に名前を付けて何度でも呼び出せるようにしたもの」です。3通りの書き方があります。本書では主に3番の**アロー関数**を使います。

```
// 1. function 宣言
function add1(a, b) {
  return a + b;
}

// 2. function 式
const add2 = function (a, b) {
  return a + b;
};

// 3. アロー関数 (=> を使う、Reactでよく使う)
const add3 = (a, b) => {
  return a + b;
};

// 3'. アロー関数の省略形 (return 1行のとき)
const add4 = (a, b) => a + b;

console.log(add1(1, 2)); // 3
console.log(add2(1, 2)); // 3
console.log(add3(1, 2)); // 3
console.log(add4(1, 2)); // 3
```

アロー関数の魅力: `function` の文字を書かなくて済むので短く、コールバック関数（あとで呼んでもらう関数）として渡すときに見やすくなります。React/Next.jsのコードはほぼアロー関数で書かれています。

0.9 配列の便利メソッド: map / filter / forEach

これは超重要です。後の章で頻繁に登場します。

```
const numbers = [1, 2, 3, 4, 5]; // 数値の配列を用意。

// forEach: 1つずつ処理する
numbers.forEach((n) => {
  console.log(n);
});

// ▼ 実行結果
// 1
// 2
// 3
// 4
// 5

// map: 1つずつ加工して新しい配列を作る
const doubled = numbers.map((n) => n * 2); // map は「各要素を加工した新しい配列を返す」
console.log(doubled); // 加工後の配列を表示。

// ▼ 実行結果
// [ 2, 4, 6, 8, 10 ]

// filter: 条件を満たす要素だけ残した新しい配列を作る
const evens = numbers.filter((n) => n % 2 === 0); // filter は「条件式が true になる要素だけ残す」
console.log(evens); // 偶数だけ残った配列を表示。

// ▼ 実行結果
// [ 2, 4 ]
```

0.10 オブジェクトの読み書き

```
const user = { name: "太郎", age: 20 }; // オブジェクトを作る。name と age の2つのプロパティ

// 値の取り出し (2つの書き方)
console.log(user.name); // "太郎" (ドット記法) // . (ドット) でプロパティ名を直接指定する書き方
console.log(user["name"]); // "太郎" (ブラケット記法) // [ ] にキー名を文字列で渡す書き方。変数で指定可能

// 値の書き換え
user.age = 21; // const で宣言されていても、オブジェクトの値は変更可能
console.log(user.age); // 21 // age が 21 に変わった。

// プロパティの追加
user.email = "taro@example.com"; // 存在しないキーに代入すると、新しいプロパティが追加される
console.log(user); // user 全体を表示すると、email が追加されていることがわかる

// ▼ 実行結果
// { name: '太郎', age: 21, email: 'taro@example.com' }

// 分割代入 (オブジェクトから一気に取り出す)
const { name, age } = user; // { } の中に取り出したいキー名を書くと、自動的に変数に代入される
console.log(name, age); // "太郎" 21
```

分割代入のコツ: React のコードでよく `const { title, author } = book;` のような書き方を見ます。これは「book オブジェクトから title と author を取り出して同名の変数を作る」省略記法です。慣れると非常に短く書けます。

0.11 セミコロン ; と改行

JavaScript/TypeScript では文末にセミコロン ; を付ける習慣があります。実は省略しても大体動きますが（自動セミコロン挿入 = Automatic Semicolon Insertion, ASI という仕組みがあるため）、本書では付ける派です（VS Code が自動で付けてくれる場合も多い）。

省略すると稀にバグる例: `return` のすぐ後で改行すると、自動で ; が入って `return undefined;` と解釈されることがあります。曖昧さを避けるため、本書では一貫して ; を付けます。

```
const a = 1; // const で a に 1 を入れる。文末の ; は「ここで1つの文が終わり」
const b = 2; // 同様に b に 2 を入れる。
console.log(a + b); // a + b は 3 になる。それを表示。
```

これで JavaScript の超基礎は OK。次の節からいよいよ TypeScript 本編に入ります。

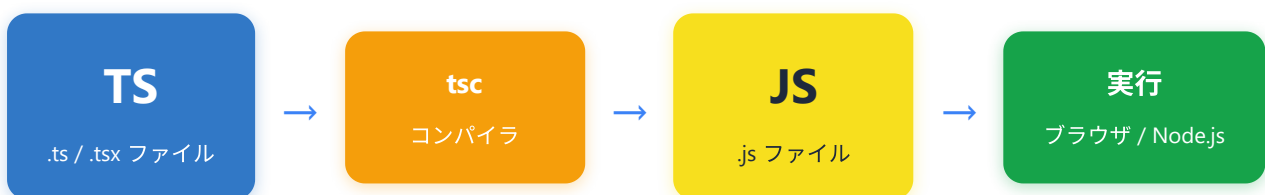
1. TypeScript とは

1.1 JavaScript との関係

TypeScript は、Microsoft（マイクロソフト：Windows や VS Code を作っている会社）が開発した **JavaScript のスーパーセット**（**superset**：上位互換。JavaScriptの機能を全て含み、さらに追加機能がある言語）です。すべての JavaScript コードは有効な TypeScript コードですが、TypeScript には「**型システム**」（Type System：タイプシステム。変数や関数に入るデータの種別を事前に定義し、チェックする仕組み）という強力な機能が追加されています。

TypeScript のコードは直接ブラウザや Node.js（ノードジェイエス：パソコン上で JavaScript を動かす実行環境）で実行できません。必ず **コンパイル**（Compile：コンパイル。人間が書いたプログラムを、コンピュータが実行できる形に変換すること。TypeScriptの場合はJavaScriptに変換する処理を指す。「トランスパイル（transpile）」とも呼ばれる）という変換処理を経て、JavaScript に変換されてから実行されます。

身近な例え： TypeScriptとJavaScriptの関係は、「下書き用紙」と「清書用紙」に似ています。TypeScript（下書き）には赤ペンで注意書き（型情報）が書いてあり、間違いがあればその場で気づけます。最終的にコンパイルすると、注意書きを消した綺麗なJavaScript（清書）ができあがります。



この流れをもう少し詳しく見てみましょう。

開発時

開発者が TypeScript でコードを書く

TypeScript コンパイラ (tsc) が型チェックを実行

型エラーがある場合 ▼

コンパイルエラー
開発者に通知

型エラーがない場合 ▼

JavaScript に変換

← 修正して再度書く

実行時

ブラウザまたは Node.js で実行

ポイント（とても重要）：TypeScript の型情報は、コンパイル後の JavaScript には一切残りません。型はあくまで「開発時の安全ネット」です。これを **型消去（Type Erasure：タイプレイジャー。型情報がコンパイルで削除される性質）** と呼びます。

例えば `const age: number = 25;` は、コンパイル後には `const age = 25;` になります。型注釈の `: number` の部分が消えるイメージです。つまり「実行時に型でエラーを止める」ことはできません。型チェックはコーディング中の VS Code と、コンパイル時の `tsc` だけが担当します。

1.2 なぜ TypeScript を使うのか

TypeScript を使う最大の理由は **型安全性** です。以下の表で、JavaScript と TypeScript の違いを比較してみましょう。

観点	JavaScript	TypeScript
型チェック	実行時にエラー発覚	コンパイル時にエラー発覚
エディタ補完	限定的	強力な自動補完
リファクタリング	手動で確認が必要	型の変更で影響範囲を自動検出
ドキュメント性	コメントに頼る	型そのものがドキュメント
バグ発見のタイミング	ユーザーが使用中に発覚	開発中に発覚
学習コスト	低い	やや高い（型の学習が必要）
大規模開発	困難	適している

1.3 JavaScript vs TypeScript の比較コード例

例1: 関数の引数に誤った型を渡すケース

```
// ---- JavaScript ----
function greet(name) {
    return "こんにちは、" + name + "さん!";
}

// 数値を渡してもエラーにならない（実行時まで気づけない）
console.log(greet(42));
// => "こんにちは、42さん!" ← 意図しない動作
```

// function 宣言。引数 name の型は推定される。
// 文字列連結で挨拶を作る。name が何であっても OK。
// 引数に数値 42 を渡しても、JavaScript は勝手に文字列に変換される。

```
// ---- TypeScript ----
function greet(name: string): string {
    return "こんにちは、" + name + "さん!";
}

// コンパイル時にエラーが発生！
console.log(greet(42));
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'
// (number 型の引数は、string 型のパラメータに代入できません)

// 正しい使い方
console.log(greet("太郎"));
// => "こんにちは、太郎さん!"
```

// : string は「型注釈 (Type Annotation)」
// ロジック自体は JavaScript と同じ
// ↑ ここで TypeScript が「string 型でなければならない」とチェックする。
// 文字列を渡すと OK。

例2: オブジェクトのプロパティアクセス

```
// ---- JavaScript ----
const user = {
  name: "田中",
  age: 25,
};

// user オブジェクトを作る。
// name プロパティに文字列。
// age プロパティに数値。 , (カンマ) でプロパティを区切る。

// タイプミスしてもエラーにならない (実行時に undefined になる)
console.log(user.nmae); // => undefined ← バグ! // name と nmae (タイプミス)。JavaScript は
```

```
// ---- TypeScript ----
const user = {
  name: "田中",
  age: 25,
};

// 型注釈を書かなくても、TypeScript が自動で推定してくれる。

// タイプミスするとコンパイル時にエラー!
console.log(user.nmae); // ↑ VS Code が即座に赤線。
// エラー: Property 'nmae' does not exist on type '{ name: string; age: number; }'.
// ('nmae' というプロパティは { name; age } 型には存在しません)
// Did you mean 'name'? // 「もしかして name のこと?」とサジェスト

// 正しいアクセス
console.log(user.name); // => "田中"
```

例3: 配列操作での型安全性

```
// ---- JavaScript ----
const numbers = [1, 2, 3, 4, 5];

// 配列リテラル。JavaScript では配列の宣言時に型を指定できない。

// 文字列を追加してもエラーにならない
numbers.push("six"); // ← バグの原因になる // push は配列末尾に要素を追加するメソッド

// 後で計算しようとするすると予期しない結果に
const sum = numbers.reduce((a, b) => a + b, 0); // reduce は配列を左から畳み込んで1つの値を返す
console.log(sum); // => "15six" ← 文字列連結になってしまう! // 数値 + 文字列 → 文字列連結のルール
```

```
// ---- TypeScript ----
const numbers: number[] = [1, 2, 3, 4, 5]; // number[] は「number 型の配列」。

// 文字列を追加しようとするとコンパイルエラー！
numbers.push("six"); // ↑ 「number しか入らない配列」に
// エラー: Argument of type 'string' is not assignable to parameter of type 'number'

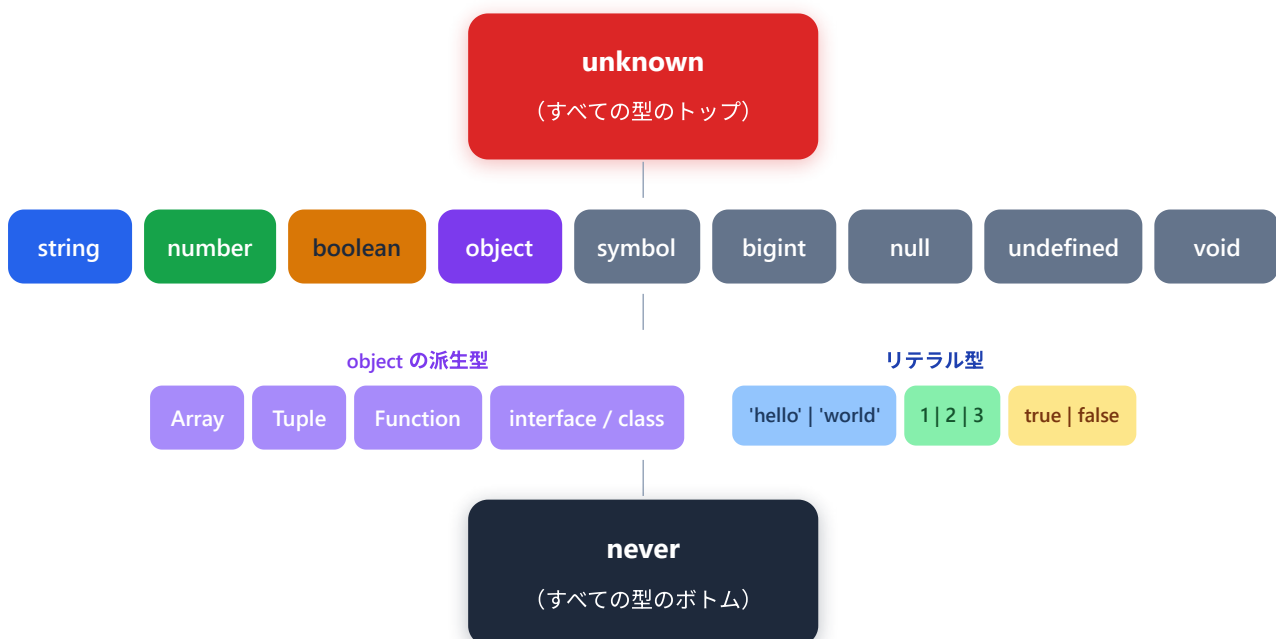
// 正しい使い方
numbers.push(6); // 6 は number なので OK。
const sum = numbers.reduce((a, b) => a + b, 0); // a, b とともに number と推論される
console.log(sum); // => 21 // 1+2+3+4+5+6 = 21
```

2. 基本的な型

TypeScript には豊富な型が用意されています。ここでは、すべての基本型を詳しく見ていきます。

TypeScript の型階層

まず、TypeScript の型システム全体を俯瞰してみましょう。



ポイント: **unknown** はすべての型の「親」で、**never** はすべての型の「子」です。**unknown** にはどんな値も代入できますが、**never** にはどんな値も代入できません。

2.1 string（文字列型）

文字列を表す型です。シングルクォート、ダブルクォート、テンプレートリテラルのいずれでも使えます。

```
// =====
// string 型のサンプル - 「文字列」を意味する型
// =====
// const は「再代入禁止の変数宣言」。一度値を入れたら別の値で上書きできない。
// 変数名の右の : string が「型注釈」。「この変数は string 型ですよ」という宣言。
// = の右側が「初期値」。

// (1) ダブルクォート " で囲んだ文字列
//      greeting という名前の string 型変数に "こんにちは" を入れる。
const greeting: string = "こんにちは";

// (2) シングルクォート ' でも書ける (JavaScript/TypeScriptでは両者同じ意味)
//      チームのコーディング規約で「" に統一」「' に統一」を決めるのが普通。
const name: string = '太郎';

// (3) テンプレートリテラル: バッククォート ` で囲み、${変数名} で値を埋め込める
//      greeting と name の中身が連結されて入る。
//      文字列連結を + で書くより読みやすい ("こんにちは" + "、" + name + "さん!" よりずっと短い)
const message: string = `${greeting}、${name}さん!`;

// (4) console.log は「ターミナルやブラウザのコンソールに値を表示する」関数
//      第1引数で渡された文字列を1行表示する。
console.log(message);
// ▼ 実行結果 (ターミナルにこう出る)
// こんにちは、太郎さん！

// (5) テンプレートリテラルは改行もそのまま含められる
//      これが文字列の中に改行コード(\n)として保持される。
const multiLine: string = `
  1行目
  2行目
  3行目
`;
console.log(multiLine);
// ▼ 実行結果
// (空行)
//   1行目
//   2行目
//   3行目
// (空行)
```

```
// =====
// string 型でエラーになる例 - VS Code が赤線で警告してくれる
// =====

// (1) 数値を string 型変数に入れようとしている → ✖
//     42 は数値リテラル。string が期待される場所には置けない。
const title: string = 42;
// ▼ エラーメッセージ
// Type 'number' is not assignable to type 'string'.
//     (number 型は string 型に代入できません)

// (2) 真偽値を string 型変数に入れようとしている → ✖
const flag: string = true;
// Type 'boolean' is not assignable to type 'string'.

// (3) null を string 型変数に入れようとしている → ✖
//     ※ tsconfig.json で "strictNullChecks": true のとき
//     null や undefined はそれぞれ独立した型として扱われる。
const nothing: string = null;
// Type 'null' is not assignable to type 'string'.
```

2.2 number（数値型）

整数・浮動小数点数の両方を含む数値型です。JavaScript と同様に、TypeScript の number はすべて浮動小数点数として扱われます。

```
// =====
// number 型のサンプル - 「数値（整数も小数も）」を意味する型
// =====

// (1) 整数
const age: number = 25;

// (2) 小数（浮動小数点数）
const price: number = 1980.5;

// (3) 負の数
const negative: number = -10;

// (4) 16進数（0x で始まる）。Web開発で色コードを書くときに使う
//      0xff の f は 15 を意味する1桁の16進文字。ff = 15*16 + 15 = 255
const hex: number = 0xff;          // 256 - 1 = 255

// (5) 2進数（0b で始まる）。フラグ操作などで使う
//      0b1010 = 1*8 + 0*4 + 1*2 + 0*1 = 10
const binary: number = 0b1010;    // = 10

// (6) 8進数（0o で始まる）。Linuxのファイル権限で見る形
//      0o744 = 7*64 + 4*8 + 4*1 = 484
const octal: number = 0o744;      // = 484

// (7) 数値リテラル区切り（_ で読みやすくする）
//      1_000_000 と書いても 1000000 と書いても結果は同じ。実行時の値に _ は含まれない。
const big: number = 1_000_000;    // = 1000000（百万）

// (8) 特殊な数値
//      Infinity = 無限大、NaN = Not a Number（不正な数値計算の結果）
const inf: number = Infinity;
const notANumber: number = NaN;

console.log(age, price, negative, hex, binary, octal, big);
// ▼ 実行結果
// 25 1980.5 -10 255 10 484 1000000

console.log(1 / 0);                // → Infinity（0で割ると無限大）
console.log(0 / 0);                // → NaN（0÷0 は数学的に未定義）
console.log("a" * 2);              // → NaN（文字列を数値として掛けると NaN）
```



```
// =====
// number 型でエラーになる例
// =====

// (1) 数字に見えるが文字列なのでダメ → ✖
const count: number = "5";
// Type 'string' is not assignable to type 'number'.

// (2) "100" + 50 は JavaScript 的には string になる ("10050"になる) → TS は型エラー扱い
const total: number = "100" + 50;
// Type 'string' is not assignable to type 'number'.
//
// ※ TypeScript はこれを書く前にエラーで止めてくれる。
//   JavaScript だけだと、後の計算で予想外のバグになる。

// (3) 数値型変数 a と文字列型変数 b を + するとどうなる?
//     → JavaScript では "10" + "20" のように文字列連結に化ける。
//     TypeScript は「これは number として使えませんよ」と止めてくれる。
const a: number = 10;
const b: string = "20";
const result: number = a + b;
// Type 'string' is not assignable to type 'number'.
```

2.3 boolean (真偽値型)

`true` または `false` のどちらかを持つ型です。

```
// =====
// boolean 型のサンプル - 「true か false」しか入らない型
// =====

// (1) 直接 true / false を入れる
const isActive: boolean = true;      // true = 「アクティブ」
const isCompleted: boolean = false;  // false = 「未完了」

// (2) 比較式の結果も boolean になる
//     >= は「以上」、=== は「等しい (型もチェック)」
//     式が成り立てば true、成り立たなければ false が返る。
const isAdult: boolean = age >= 18;   // age が 25 なので true
const isEmpty: boolean = name === ""; // name が "太郎" なので false

// (3) 論理演算子 (&& と ||)
//     && (AND) : 両方 true なら true、片方でも false なら false
//     || (OR)  : 片方でも true なら true、両方 false なら false
//     ! (NOT)  : true ⇄ false を反転
const canAccess: boolean = isActive && isAdult; // true && true = true

console.log(isActive, isCompleted, isAdult, isEmpty, canAccess);
// ▼ 実行結果
// true false true false true
```

```
// =====
// boolean 型でエラーになる例
// =====

// (1) 数字 1 は「真っぽい値」だが boolean 型ではない → ✖
const flag: boolean = 1;
// Type 'number' is not assignable to type 'boolean'.

// (2) "true" は文字列であって、真偽値ではない → ✖
const active: boolean = "true";
// Type 'string' is not assignable to type 'boolean'.

// (3) 0 も「偽っぽい値」だが boolean 型ではない → ✖
//     JavaScript では if (0) {...} は実行されない (falsy扱い) が、
//     boolean 型としては number の 0 と boolean の false は別物。
const truthy: boolean = 0;
// Type 'number' is not assignable to type 'boolean'.
```

2.4 array (配列型)

同じ型の要素を複数格納するコレクションです。書き方は2通りあります。

```
// =====
// 配列型のサンプル - 「同じ型の値を順番に並べたリスト」
// =====

// ----- 書き方1: 型名 + [] -----
// string[] は「string が並んだ配列」の意味。一番一般的な書き方。

// (1) 文字列の配列
const fruits: string[] = ["りんご", "みかん", "バナナ"];
//                               [0]       [1]       [2]   ← 添え字 (0から始まる)

// (2) 数値の配列
const scores: number[] = [85, 90, 78, 92];

// (3) 真偽値の配列
const flags: boolean[] = [true, false, true];

// ----- 書き方2: Array<型名> -----
// 結果は [] 記法と同じ。読み手の好みで選ぶ。

const colors: Array<string> = ["赤", "青", "緑"];
const ages: Array<number> = [20, 25, 30];

// ----- 配列の主な操作 -----

// (4) push: 配列の末尾に要素を追加 (破壊的に変更される)
//       型が一致する値だけ受け付ける (string配列に number は入らない)
fruits.push("ぶどう");           // fruits = ["りんご", "みかん", "バナナ", "ぶどう"]
scores.push(100);                // scores = [85, 90, 78, 92, 100]

// (5) [添え字] でアクセス。存在しない添え字は undefined (厳密モードでは型エラー)
const first: string = fruits[0]; // = "りんご"
console.log(first);
// ▼ 実行結果
// りんご

// (6) 配列の長さを取る
console.log(fruits.length);
// ▼ 実行結果
// 4

// (7) 空配列を作るときも型注釈を書いておくのが安全
//       こうしないと never[] と推論されてしまい、後で push できない。
const emptyStrings: string[] = [];
emptyStrings.push("hello");      // OK
console.log(emptyStrings);
```

```
// ▼ 実行結果
// [ 'hello' ]
```

```
// --- エラーになる例 ---
const numbers: number[] = [1, 2, "three"]; // number[] と宣言しているのに "
// エラー: Type 'string' is not assignable to type 'number'

const items: string[] = ["a", "b", "c"]; // string 専用配列。
items.push(42); // 42 は number なので入れられな
// エラー: Argument of type 'number' is not assignable to parameter of type 'string'

// 異なる型が混在する配列を number[] として定義
const mixed: number[] = [1, true, "hello"]; // true (boolean) も "hello"
// エラー: Type 'boolean' is not assignable to type 'number'
// エラー: Type 'string' is not assignable to type 'number'
```

2.5 tuple (タプル型)

タプル (tuple: 複数の値を順序付きで束ねた、長さが固定のデータ構造) 型 は、固定長で、各位置の型が決まっている配列 です。通常の配列と異なり、要素ごとに異なる型を持てます。

```
// --- 正しい例 ---

// [名前, 年齢] のタプル
const person: [string, number] = ["田中", 30]; // 型注釈の [string, number] が

// [ID, 名前, アクティブフラグ] のタプル
const record: [number, string, boolean] = [1, "太郎", true]; // 3要素のタプル。位置ごとに型が決

// 要素へのアクセス (型が正しく推論される)
const personName: string = person[0]; // "田中" ← string 型 // [0] は1番目の要素を取り出す (配
const personAge: number = person[1]; // 30 ← number 型 // [1] は2番目。number と推論され

// 分割代入も可能
const [name, age] = person; // 配列形式の分割代入。person[0]
// name は string 型、age は number 型

// オptionalなタプル要素
const optionalTuple: [string, number?] = ["田中"]; // 2要素目に ? を付けると「省略し
// 2番目の要素はあってもなくてもよい
```

```
// --- エラーになる例 ---
const pair: [string, number] = [42, "田中"];           // 1番目が string でなければなら
// エラー: Type 'number' is not assignable to type 'string' (1番目)
// エラー: Type 'string' is not assignable to type 'number' (2番目)

const triple: [string, number, boolean] = ["太郎", 25]; // タプル型は「長さも固定」。3要素
// エラー: Source has 2 element(s) but target requires 3

// 型が異なる位置へのアクセス
const data: [string, number] = ["hello", 42];          // 1番目が string、2番目が numb
const wrongType: number = data[0];                    // [0] は stringなのに numbe
// エラー: Type 'string' is not assignable to type 'number'
```

2.6 object (オブジェクト型)

キーと値のペアを持つ構造化されたデータ型です。

```

// --- 正しい例 ---

// オブジェクトリテラル型
const user: { name: string; age: number; email: string } = { // { ... } 中の「キー: 型」の
  name: "田中太郎", // string プロパティ。
  age: 30, // number プロパティ。
  email: "tanaka@example.com", // string プロパティ。
};

// オptionalプロパティ (? をつける)
const product: { name: string; price: number; description?: string } = { // description?
  name: "TypeScript入門書",
  price: 2980,
  // description は省略可能 // description は書かなくても
};

// 読み取り専用プロパティ
const config: { readonly apiUrl: string; readonly timeout: number } = { // readonly キー
  apiUrl: "https://api.example.com",
  timeout: 5000,
};

// ネストしたオブジェクト
const company: { // 型注釈の中にも { ... }
  name: string;
  address: { // address 自体がオブジェク
    city: string;
    zipCode: string;
  };
} = {
  name: "サンプル株式会社",
  address: {
    city: "東京",
    zipCode: "100-0001",
  },
};

```

```
// --- エラーになる例 ---
const user: { name: string; age: number } = { // age が必須なのに...
  name: "田中",
  // age がない! // 必須プロパティ漏れはエラー。
};
// エラー: Property 'age' is missing in type '{ name: string; }'

const item: { name: string; price: number } = { // 型で許可されているのは name と price
  name: "ペン",
  price: 100,
  color: "赤", // 余分なプロパティ // 余計なプロパティ (型定義に無いキー)
};
// エラー: Object literal may only specify known properties,
// and 'color' does not exist in type '{ name: string; price: number; }'

// readonly プロパティへの再代入
const settings: { readonly theme: string } = { theme: "dark" }; // readonly なので theme は変更不可
settings.theme = "light"; // 後から書き換えようとするとうるエラー
// エラー: Cannot assign to 'theme' because it is a read-only property
```

2.7 any 型

すべての型チェックを無効化する 型です。どんな値でも受け入れ、どんな操作も許可します。

```
// --- any の使用例 (非推奨だが動く) ---
let anything: any = "文字列"; // any は「何でもアリ型」。型チェックを全部スキップする魔法のキーワード
anything = 42; // OK (number を代入) // どんな型の値も再代入できる。
anything = true; // OK (boolean を代入) // ↑同じく。
anything = [1, 2, 3]; // OK (配列を代入) // ↑同じく。
anything.foo(); // OK (存在しないメソッドも呼べる → 実行時エラー!) // foo というメソッドが存在しない
anything.bar.baz; // OK (存在しないプロパティもアクセス可能 → 実行時エラー!) // .bar が存在しない
```

警告: `any` を使うと TypeScript を使う意味がほぼなくなります。原則として `any` は使わないでください。やむを得ない場合 (外部ライブラリの型定義がない場合など) のみ、限定的に使います。

```
// --- なぜ any が危険か ---
function processData(data: any) { // 引数 data の型を any に
  // 型チェックが一切行われない
  return data.toUpperCase(); // data が string でなければ実行時エラー // .toUpperCase() は s
}

processData("hello"); // OK: "HELLO" // 文字列なので OK。
processData(42); // 実行時エラー: data.toUpperCase is not a function // 数値に toUp
processData(null); // 実行時エラー: Cannot read properties of null // null の .to
```

2.8 unknown 型

any の安全な代替です。どんな値も代入できますが、使う前に型チェックが必要です。

```
// --- unknown の正しい使い方 ---
let value: unknown = "こんにちは"; // unknown 型変数を宣言。「中身は何か
value = 42; // OK (代入は自由) // どんな型でも代入はできる (ここは any
value = true; // OK

// ただし、そのまま使うことはできない
// const upper: string = value.toUpperCase(); // ↑ unknown のままだとメソッド呼び出
// エラー: 'value' is of type 'unknown'

// 型チェック (型ガード) を行ってから使う
if (typeof value === "string") { // typeof は値の型を文字列で返す演算
  // このブロック内では value は string 型として扱える // 条件式により TypeScript が「ここでは
  console.log(value.toUpperCase()); // OK // string なので toUpperCase が呼
}

if (typeof value === "number") {
  // このブロック内では value は number 型として扱える
  console.log(value.toFixed(2)); // OK // .toFixed(2) は number のメソッ
}
```



```
// --- any と unknown の比較 ---

// any: 危険 (型チェックなし)
function unsafeProcess(data: any): string {
    return data.toUpperCase(); // 実行時に壊れる可能性あり
}

// unknown: 安全 (型チェック必須)
function safeProcess(data: unknown): string {
    if (typeof data === "string") {
        return data.toUpperCase(); // 型チェック済みなので安全
    }
    return "不明なデータ";
}

// 引数を any にすると...
// 何でも通るが安全性ゼロ。

// unknown にすると...
// 必ず型ガードを書かないと中身を
// string と確定したブロックなので
// 型が string でなければデフォルト
```

2.9 never 型

決して発生しない値の型です。主に以下の場面で使われます。

```

// --- 用途1: 絶対に値を返さない関数 ---
function throwError(message: string): never {
    throw new Error(message);
    // この関数は必ず例外を投げるので、正常に値を返すことがない
}

// --- 用途2: 無限ループ ---
function infiniteLoop(): never {
    while (true) {
        // 永遠に終わらない
    }
}

// --- 用途3: 到達不可能なコードの検出 (網羅性チェック) ---
type Shape = "circle" | "square" | "triangle";

function getArea(shape: Shape): number {
    switch (shape) {
        case "circle":
            return Math.PI * 10 * 10;
        case "square":
            return 10 * 10;
        case "triangle":
            return (10 * 10) / 2;
        default:
            // すべてのケースを処理済みなら、shape は never 型になる
            const _exhaustiveCheck: never = shape;
            return _exhaustiveCheck;
    }
}

```

// 戻り値の型 : never は「決して返らない」ことを示す型。
 // throw = 例外を投げる文。never 型に型チェックが通る。

// ずっと終わらない関数も never 型になる。
 // while (true) は「true のまま無限ループ」を示す。

// ユニオン型で3種類のリテラルを定義する。
 // shape を受け取って面積を返す関数。
 // switch 文: 値ごとに分岐。
 // shape が "circle" のとき、Math.PI は円周率 π 。半径10の円の面積。
 // 1辺10の正方形の面積。
 // 底辺10・高さ10の三角形の面積。
 // 上の case で全てのリテラルを処理済みなら、shape は never 型になる。
 // never 型に shape を代入すると型エラー。
 // never は number にも代入可能。

```
// --- never 型の重要性: 網羅性チェック ---

// Shape に新しい種類を追加した場合
type Shape = "circle" | "square" | "triangle" | "pentagon"; // pentagon を追加 // ↑ あとか

function getArea(shape: Shape): number {
  switch (shape) {
    case "circle":
      return Math.PI * 10 * 10;
    case "square":
      return 10 * 10;
    case "triangle":
      return (10 * 10) / 2;
    default:
      // "pentagon" のケースが未処理なのでエラーになる! // default に来る可能性が「pentagon」
      const _exhaustiveCheck: never = shape; // "pentagon" は never に属しない
      // エラー: Type 'string' is not assignable to type 'never'
      // → "pentagon" の処理を追加し忘れたことに気づける
      return _exhaustiveCheck;
  }
}
```

2.10 void 型

値を返さない関数の戻り値の型です。`never` と違い、関数自体は正常に終了します。

```
// --- 正しい例 ---

function logMessage(message: string): void { // 戻り値: void = 「値を返さない」
  console.log(message); // 文字列を表示するだけ。
  // return 文がない、または return; のみ // 暗黙的に undefined を返している
}

function showAlert(text: string): void {
  alert(text); // alert はブラウザのポップアップ
  return; // return は OK (値を返さない) // return の後に値を書かなければならない
}

// void 型の変数には undefined のみ代入可能
const result: void = undefined; // void 変数に入れられるのは undefined のみ
```

```
// --- エラーになる例 ---
function greet(name: string): void {
    return `こんにちは、${name}さん`;
    // エラー: Type 'string' is not assignable to type 'void'
    // void なんのに値を返している
}

const value: void = "hello";
// エラー: Type 'string' is not assignable to type 'void'

const num: void = 42;
// エラー: Type 'number' is not assignable to type 'void'

// void と宣言しているのに...
// 文字列を return しているの

// void に文字列を入れられない

// void に数値も入れられない。
```

2.11 null と undefined

null（ヌル：明示的な「無」）は「値が意図的に空」であることを表し、**undefined**（アンディファインド：未定義）は「値が未定義」であることを表します。

使い分けの目安: - **undefined** : 自然発生する「値がない」。変数を宣言しただけで初期化していない、関数何も return しないとき、オブジェクトの存在しないプロパティを読んだとき。 - **null** : プログラムが「ここは意図的に空ですよ」と明示するときに使う。例えば API で「ユーザーが見つからない」場合に **null** を返すなど。 - **迷ったら undefined を使う** のが TypeScript の流儀。**null** は外部から値が来る場合（API, DB など）に限定するのが一般的。

```
// --- 正しい例 ---

// strictNullChecks が有効な場合 (推奨)
let nullableString: string | null = null; // 明示的に null を許可 // string | null は「string または null」
let optionalValue: number | undefined = undefined; // number または undefined

// null チェック
function findUser(id: number): string | null { // 戻り値が「string または null」
  if (id === 1) { // === は厳密等価
    return "田中太郎";
  }
  return null; // ユーザーが見つからない場合 // 1 以外は明示的に null
}

const user = findUser(999); // user の型は string | null
if (user !== null) { // null でないか
  console.log(user.toUpperCase()); // null チェック後なので安全 // このブロック内では user は null ではない
}

// オプショナルチェイニング (?.)
const length = user?.length; // user が null なら undefined を返す // ?. は「左が null なら undefined を返す」

// null 合体演算子 (??)
const displayName = user ?? "ゲスト"; // user が null/undefined なら "ゲスト" // ?? は「左が null/undefined なら右の値を返す」
```

```
// --- エラーになる例 (strictNullChecks 有効時) ---

let name: string = null; // string 型に null を代入しようとしている
// エラー: Type 'null' is not assignable to type 'string'

let age: number = undefined; // number 型に undefined を代入しようとしている
// エラー: Type 'undefined' is not assignable to type 'number'

// null チェックなしでのメソッド呼び出し
function findItem(id: number): string | null {
  return id > 0 ? "アイテム" : null; // 三項演算子 ?
}

const item = findItem(-1); // item の型は string | null
console.log(item.toUpperCase()); // チェックなしでメソッド呼び出し
// エラー: 'item' is possibly 'null'
// → item が null の可能性があるため、チェックなしでは使えない
```

基本型のまとめ表

型	説明	例	使用場面
<code>string</code>	文字列	<code>"hello"</code> , <code>'world'</code>	テキストデータ全般
<code>number</code>	数値	<code>42</code> , <code>3.14</code>	数値計算、ID
<code>boolean</code>	真偽値	<code>true</code> , <code>false</code>	フラグ 条件
<code>string[]</code>	文字列配列	<code>["a", "b"]</code>	リストデータ
<code>[string, number]</code>	タプル	<code>["太郎", 25]</code>	固定構造のペア
<code>object</code>	オブジェクト	<code>{ key: "value" }</code>	構造化データ
<code>any</code>	なんでも	すべて	使用非推奨
<code>unknown</code>	不明な型	すべて	安全な <code>any</code> の代替
<code>never</code>	発生しない	なし	網羅性チェック
<code>void</code>	返り値なし	<code>undefined</code>	関数の戻り値
<code>null</code>	空	<code>null</code>	意図的な空値
<code>undefined</code>	未定義	<code>undefined</code>	未初期化の値

3. 型注釈と型推論

3.1 明示的な型注釈

型注釈（Type Annotation：タイプアノテーション。変数や関数の引数・戻り値に「これは何の型か」を明示的に書くこと）とは、変数や関数のパラメータ・戻り値に明示的に型を記述することです。コロンの（`:`）の後に型を書きます。

```
// 変数の型注釈
const bookTitle: string = "TypeScript入門"; // 変数名 : 型 = 値 の
const pageCount: number = 350; // : number で「数値
const isPublished: boolean = true; // : boolean で「真

// 関数のパラメータと戻り値の型注釈
function calculateTotal(price: number, quantity: number): number { // (引数1: 型, 引数2
    return price * quantity; // price と quanti
}

// アロー関数の型注釈
const add = (a: number, b: number): number => a + b; // アロー関数版。( ):

// オブジェクトの型注釈
const book: { title: string; author: string; pages: number } = { // { キー: 型; ...
    title: "TypeScript入門",
    author: "山田太郎",
    pages: 350,
};

// 配列の型注釈
const tags: string[] = ["プログラミング", "TypeScript", "入門"]; // string[] = 「string
```

3.2 型推論の仕組み

TypeScript は非常に賢い ****型推論**** (Type Inference: タイプインファレンス。初期値や使われ方から Type

```
```typescript
// --- TypeScript が自動で型を推論する例 ---

// 変数の初期化から推論
const message = "こんにちは"; // string と推論 // 右辺が文字列リテラル
const count = 42; // number と推論 // 数値リテラル → nu
const isValid = true; // boolean と推論 // true/false → boo
const items = [1, 2, 3]; // number[] と推論 // 配列の中身が全部 nu

// 関数の戻り値も推論される
function multiply(a: number, b: number) { // 戻り値の型注釈を
 return a * b; // 戻り値は number と推論される // a * b が number な
}

// オブジェクトの構造も推論される
const user = {
 name: "田中", // name: string // 右辺の値から各プロ
 age: 25, // age: number
 isActive: true, // isActive: boolean
};
// user の型は { name: string; age: number; isActive: boolean } と推論
```

```
// 配列のメソッドから推論
const doubled = [1, 2, 3].map((n) => n * 2); // map の引数 n は number
// doubled は number[] と推論

// 条件式から推論
const status = count > 10 ? "many" : "few"; // 三項演算子 ? :
// status は string と推論
```

// --- 型推論の注意点 ---

```
// let で宣言すると広い型に推論される
let color = "red"; // string と推論 ("red" リテラル型ではない) // let は後から再代入可能
color = "blue"; // OK (string なので他の文字列も代入可能)

// const で宣言するとリテラル型に推論される
const direction = "north"; // "north" と推論 (リテラル型) // const は変わらない
// direction = "south"; // エラー: const なので再代入不可

// 空配列は any[] に推論される (要注意)
const emptyList = []; // any[] と推論 // 中身がないので
// → 型注釈をつけるべき

const emptyStrings: string[] = []; // 明示的に型を指定 // 必ず型注釈を書くの
```

### 3.3 いつ型注釈を書くべきか





## 型注釈を書くべき場面

```
// 1. 関数のパラメータ (必須! 推論できない)
function greet(name: string, age: number): string {
 return `${name}さんは${age}歳です`;
}

// 2. 空配列を初期化する場合
const users: string[] = []; // 型注釈がないと any[] になる

// 3. 関数の戻り値を明確にしたい場合 (公開APIなど)
function fetchUser(id: number): Promise<User> {
 // 戻り値の型が明確になり、実装ミスを防げる
 return fetch(`/api/users/${id}`).then((res) => res.json());
}

// 4. 複数の型を受け入れる場合
let result: string | number;
result = "成功";
result = 404;

// 5. オブジェクトの構造を明確にしたい場合
interface Config {
 apiUrl: string;
 timeout: number;
 retries: number;
}

const config: Config = {
 apiUrl: "https://api.example.com",
 timeout: 5000,
 retries: 3,
};
```

## 型推論に任せてよい場面

```
// 1. 初期値から型が明らかな場合
const name = "田中太郎"; // 明らかに string // : string と書かなくて
const age = 30; // 明らかに number
const isActive = true; // 明らかに boolean

// 2. 配列リテラルで初期化する場合
const fruits = ["りんご", "みかん"]; // 明らかに string[] // 中身があれば推論でき

// 3. 関数の戻り値が単純な場合
function add(a: number, b: number) { // 戻り値の型を省略。
 return a + b; // 明らかに number を返す // number + number →
}

// 4. 変数の型が変わらない場合
const total = price * quantity; // 明らかに number // 計算結果から推論。
```

## 4. インターフェースと型エイリアス

### 4.1 interface の定義方法

**interface** はオブジェクトの「形（シェイプ）」を定義する方法です。

```
// =====
// interface の基本 - 「オブジェクトの設計図」を作る
// =====
// interface は「このオブジェクトは こういう形をしてないとダメだよ」という
// ルールを宣言する仕組み。クラスの設計図のようなものを軽量に書ける。
// 大文字始まりにするのが慣習（クラス名と同じ）。

// (1) User という名前の interface を定義
// 「User 型のオブジェクトには id, name, email, age の4つが必須」と宣言
interface User {
 id: number; // 数値型のID（必須）
 name: string; // 文字列型の名前（必須）
 email: string; // 文字列型のメールアドレス（必須）
 age: number; // 数値型の年齢（必須）
}

// (2) User 型の変数を作る
// プロパティが1つでも欠けるとエラー、余計なプロパティもエラー（厳格チェック）
const user: User = {
 id: 1,
 name: "田中太郎",
 email: "tanaka@example.com",
 age: 30,
};

console.log(user);
// ▼ 実行結果
// { id: 1, name: '田中太郎', email: 'tanaka@example.com', age: 30 }

// =====
// オptionalプロパティ「?」を使うと、あってもなくても良くなる
// =====

interface Product {
 id: number; // 必須
 name: string; // 必須
 price: number; // 必須
 description?: string; // ← 「?」 付きなので省略可能。値が無いときは undefined
}

// description を省略しても OK
const pen: Product = {
 id: 1,
 name: "ボールペン",
 price: 150,
 // description は書いていないが、? なのでエラーにならない
};
```

```

console.log(pen.description);
// ▼ 実行結果
// undefined

// =====
// readonly: 「あとから変えられないプロパティ」
// =====

interface Config {
 readonly apiUrl: string; // readonly = 後で代入禁止
 readonly timeout: number;
}

const config: Config = {
 apiUrl: "https://api.example.com",
 timeout: 5000,
};

// config.apiUrl = "https://other.com"; // ← これを書くとエラー
// ▼ エラー
// Cannot assign to 'apiUrl' because it is a read-only property.
// ('apiUrl' は読み取り専用プロパティのため代入できません)

```

```
// =====
// interface の継承 (extends) - 既存の型に項目を追加した新しい型を作る
// =====

// (1) ベースとなる Animal 型
interface Animal {
 name: string; // 動物の名前
 age: number; // 年齢
}

// (2) Dog は Animal を「extends」している
// → Animal の name, age を全て持ったうえで、breed と isVaccinated が追加される。
// 継承元の項目は書かなくてもOK (自動で引き継がれる)。
interface Dog extends Animal {
 breed: string; // 犬種 (Dog で追加)
 isVaccinated: boolean; // ワクチン接種済みか (Dog で追加)
}

// (3) Dog 型のオブジェクトには合計4つのプロパティが必須
const myDog: Dog = {
 name: "ポチ", // Animal 由来
 age: 3, // Animal 由来
 breed: "柴犬", // Dog 固有
 isVaccinated: true, // Dog 固有
};
console.log(myDog.name, myDog.breed);
// ▼ 実行結果
// ポチ 柴犬

// =====
// 複数の interface を同時に継承 (カンマ区切り)
// =====
// 「ID を持つ」「タイムスタンプを持つ」のような共通の特徴を細かく分けて、
// 必要な組み合わせを extends で混ぜるのが現代的な設計。

// (1) 「ID を持つ」だけを表現する interface
interface HasId {
 id: number;
}

// (2) 「作成日時・更新日時を持つ」だけを表現する interface
interface HasTimestamp {
 createdAt: Date; // Date は JavaScript 標準の日時オブジェクト
 updatedAt: Date;
}

// (3) BlogPost は HasId と HasTimestamp の両方を継承し、独自フィールドを追加
interface BlogPost extends HasId, HasTimestamp {
```

```

title: string; // タイトル
content: string; // 本文
author: string; // 著者
}

// (4) BlogPost 型のオブジェクトには6つのプロパティが必須
// new Date("2025-01-01") は「2025年1月1日 0:00 UTC」を表す Date オブジェクト
const post: BlogPost = {
 id: 1,
 createdAt: new Date("2025-01-01"),
 updatedAt: new Date("2025-06-15"),
 title: "TypeScript入門",
 content: "TypeScriptの基礎を学びましょう",
 author: "田中太郎",
};

console.log(post.title, "投稿日:", post.createdAt.toISOString());
// ▼ 実行結果
// TypeScript入門 投稿日: 2025-01-01T00:00:00.000Z

```

```

// interface でメソッドを定義
interface Calculator {
 add(a: number, b: number): number;
 subtract(a: number, b: number): number;
 multiply(a: number, b: number): number;
 divide(a: number, b: number): number;
}

const calc: Calculator = {
 add: (a, b) => a + b,
 subtract: (a, b) => a - b,
 multiply: (a, b) => a * b,
 divide: (a, b) => {
 if (b === 0) throw new Error("0で割ることはできません");
 return a / b;
 },
};

```

// 「メソッド (関数プロパティ)」  
 // add メソッドの型。引数2つ、戻り値 number。  
 // 同様に subtract (引き算)、multiply (掛け算)、divide (割り算)。  
 // Calculator 型として4つのメソッドを定義。  
 // 各メソッドはアロー関数で実現。  
 // 割り算は 0 除算チェックが必要。  
 // b が 0 のとき例外を投げる。  
 // 正常時は割った結果を返す。

## 4.2 type の定義方法

**type** (型エイリアス) はあらゆる型に名前をつける方法です。

```

// 基本的な type エイリアス
type UserName = string;
type Age = number;
type IsActive = boolean;

// type で型に別名を付ける。
// Age は number の別名。
// IsActive = boolean。

// オブジェクト型
type User = {
 id: number;
 name: string;
 email: string;
 age: number;
};
// type でオブジェクト型を

const user: User = {
 id: 1,
 name: "田中太郎",
 email: "tanaka@example.com",
 age: 30,
};
// User 型として変数を作る

// ユニオン型 (interface ではできない)
type Status = "active" | "inactive" | "pending";
type Id = string | number;
// 文字列リテラルのユニオン
// string か number のど

const userStatus: Status = "active";
const userId: Id = "user_123";
// "active" は許可された値
// string が許可されている

// タプル型
type Coordinate = [number, number];
type NameAge = [string, number];
// 2要素タプル。[緯度, 経度]
// [名前, 年齢] のタプル。

const point: Coordinate = [35.6762, 139.6503]; // 東京の座標
// Coordinate 型として代入

// 関数型
type MathOperation = (a: number, b: number) => number;
// 「(a: number, b: number) => number」

const add: MathOperation = (a, b) => a + b;
const subtract: MathOperation = (a, b) => a - b;
// MathOperation 型として宣言
// 同じ型から別の関数を作る

```

```

// 交差型 (Intersection Type)
type HasName = { // 「name を持つ」だけを定義
 name: string;
};

type HasAge = { // 「age を持つ」だけを定義
 age: number;
};

type Person = HasName & HasAge; // & は交差型 (intersection)
// Person は { name: string; age: number; } と同じ

const person: Person = {
 name: "田中",
 age: 30,
};

// 条件付き型 (Conditional Type)
type IsString<T> = T extends string ? "yes" : "no"; // T が string 型に含まれる ?

type A = IsString<string>; // "yes" // T = string → "yes"。
type B = IsString<number>; // "no" // T = number → "no"。

// マップ型 (Mapped Type)
type Readonly<T> = { // T の全プロパティを readonly にする
 readonly [P in keyof T]: T[P]; // [P in keyof T] は keyof T の型
};

type ReadonlyUser = Readonly<User>; // User の全プロパティが readonly になる
// すべてのプロパティが readonly になる

```



## 4.3 interface vs type の使い分け

機能	interface	type
オブジェクトの形を定義	OK	OK
継承 (extends)	OK	交差型 ( & ) で代替
実装 (implements)	OK	OK
宣言のマージ (Declaration Merging)	OK	不可
ユニオン型	不可	OK
タプル型	不可	OK
プリミティブ型のエイリアス	不可	OK
マップ型	不可	OK
条件付き型	不可	OK
計算されたプロパティ	不可	OK

```
// --- interface でしかできないこと: 宣言のマージ ---
interface Window {
 title: string;
}

interface Window {
 appVersion: number;
}

// 2つの宣言がマージされる
// Window = { title: string; appVersion: number; }

// type では同じ名前でも再定義するとエラーになる
type Animal = { name: string };
type Animal = { age: number }; // エラー: Duplicate identifier 'Animal'
```

// 同じ名前の interface

// 結果は両方のプロパティ

// 1つ目の定義。

// 同じ名前でも再定義

```
// --- type でしかできないこと ---

// ユニオン型
type Result = "success" | "error" | "loading"; // | で複数のリテラルを

// プリミティブのエイリアス
type ID = string | number; // プリミティブ型に別名

// タプル型
type Point = [number, number]; // タプル型のエイリアス

// 関数型
type Formatter = (input: string) => string; // 関数型のエイリアスも

// 条件付き型
type NonNullable<T> = T extends null | undefined ? never : T; // 条件付き型も type
```

使い分けの指針:

- **interface を使う場面:** オブジェクトの形状を定義する場合（特にクラスが実装する場合や、ライブラリの型を拡張したい場合）
- **type を使う場面:** ユニオン型、タプル型、関数型、プリミティブ型エイリアスなど、interface では表現できない型を定義する場合

**実務でのコツ:** チーム内で統一するのが最も重要です。迷ったら **interface** をデフォルトで使い、**interface** で表現できない場合のみ **type** を使う という方針が一般的です。

## 4.4 書籍管理アプリで使う型の定義例

実際のアプリケーション開発を想定して、書籍管理アプリに必要な型を定義してみましょう。初心者でもわかるように、ほぼ全行に解説コメントを付けています。

```
// =====
// 書籍管理アプリの型定義（詳細コメント版）
// =====
// このファイルはアプリ全体で使う「データの形」を集めた辞書のようなもの。
// ここで型を1度作っておくと、関数の引数や useState の中身など
// あらゆる場所で同じ型を再利用でき、間違いを TypeScript が指摘してくれる。
// =====

// -----
// (1) 「読書ステータス」の型
// -----
// 「ユニオン型」(| で区切って列挙)を使い、値を3つの文字列のどれかに限定する。
// こうすると status = "yomu" のような typo が即座にエラーになる。
type ReadingStatus = "want-to-read" | "reading" | "completed";
// ↑読みたい ↑読書中 ↑読了

// -----
// (2) 「評価」の型 - 1~5 の数値リテラルだけを許す
// -----
// number だけだと 0 や 100 も入ってしまう。1~5 のいずれかに限定したいので、
// 数値リテラル型のユニオンで宣言する。
type Rating = 1 | 2 | 3 | 4 | 5;

// -----
// (3) カテゴリの型 - 列挙したいときの定石
// -----
// 各値の右側のコメントは「人間用のメモ」。実行時には消える。
type BookCategory =
 | "fiction" // 小説
 | "non-fiction" // ノンフィクション
 | "technology" // 技術書
 | "business" // ビジネス
 | "self-help" // 自己啓発
 | "other"; // その他

// -----
// (4) 著者情報の型 - interface で「オブジェクトの形」を定義
// -----
// `?` を付けたプロパティは「あってもなくてもいい」（オプション）
interface Author {
 id: string; // 著者のユニークID（必須）
 name: string; // 著者名（必須）
 nationality?: string; // 国籍（省略可能。? が付く）
}
```

```
// -----
// (5) 書籍の基本情報
// -----
// `author` プロパティは前で定義した Author 型を再利用している。
// このように型を入れ子にできるのが TypeScript の便利なところ。
interface Book {
 id: string; // 書籍のユニークID
 title: string; // タイトル
 author: Author; // 著者 (Author 型のオブジェクト)
 isbn: string; // ISBN番号 (書籍の世界共通ID)
 category: BookCategory; // カテゴリ (前述の6種類のどれか)
 pages: number; // 総ページ数
 publishedDate: string; // 出版日。ISO 8601 形式 "2025-01-15"
 coverImageUrl?: string; // 表紙画像URL (省略可能)
 description?: string; // 説明文 (省略可能)
}

// -----
// (6) 読書記録の型 - Book を「継承」して項目を追加
// -----
// `extends Book` は「Book のすべての項目をそのまま受け継いだうえで、追加項目を持つ」
// という意味。継承を使うと「Book にこれだけ足したやつ」という意図が明確になる。
interface ReadingRecord extends Book {
 status: ReadingStatus; // 読書ステータス (必須)
 rating?: Rating; // 評価 (読了後にのみ設定するためオプショナル)
 startDate?: string; // 読み始めた日
 endDate?: string; // 読み終わった日
 notes?: string; // メモ
 currentPage?: number; // 現在のページ (読書中のみ意味を持つ)
}

// -----
// (7) ユーティリティ型 Omit / Partial の活用
// -----
// 同じことを毎回手書きすると面倒なので、TypeScript の組み込みユーティリティを使う。

// `Omit<T, K>` = T から K プロパティを抜いた型を作る。
// 新規作成時は id がまだ存在しないので、Book から id を抜く。
type CreateBookInput = Omit<Book, "id">;
// 結果として「id 以外の全フィールドが必須」の型ができる。

// `Partial<T>` = T のすべてのプロパティを「オプショナル」にした型。
// 更新時は「変えたい項目だけ」送れば良いので、全項目を ? 付きにしたい。
type UpdateBookInput = Partial<Omit<Book, "id">>;
// 結果として「id 以外の全フィールドが省略可」の型ができる。
```

```
// -----
// (8) フィルター条件の型 - 検索・並び替えのオプション
// -----
// すべて省略可 (?) にして、ユーザーが指定したものだけ使うパターン。
interface BookFilter {
 status?: ReadingStatus; // ステータスで絞る
 category?: BookCategory; // カテゴリで絞る
 searchQuery?: string; // タイトルまたは著者名で検索
 sortBy?: "title" | "publishedDate" | "rating"; // 並び替え基準
 sortOrder?: "asc" | "desc"; // asc=昇順 (1→9, A→Z), desc=降順 (9→1, Z→A)
}

// -----
// (9) ジェネリクスを使った API レスポンス型
// -----
// `<T>` は「あとで決まる型のプレースホルダー」。
// 例えば `ApiResponse<Book>` と書けば T = Book になり、
// `ApiResponse<Author>` と書けば T = Author になる。
// 「成功・失敗の枠組みは同じだが、データの中身は呼び出し側で決まる」を表現するのに最適。
interface ApiResponse<T> {
 data: T; // 取得したデータ本体 (型は呼び出し側次第)
 success: boolean; // 成否
 message?: string; // メッセージ (省略可)
}

// -----
// (10) ページネーション付きレスポンス
// -----
// 一覧取得APIは「データ配列+ページ情報」をセットで返すのが定番。
// data: T[] の T[] は「T の配列」を意味する記法。
interface PaginatedResponse<T> {
 data: T[]; // 1ページ分のデータ配列
 total: number; // 全件数
 page: number; // 現在のページ番号 (1始まり)
 perPage: number; // 1ページあたりの件数
 totalPages: number; // 総ページ数
}

// ===== 使用例 =====

// 書籍の作成
const newBook: CreateBookInput = {
 title: "TypeScript実践ガイド",
 author: {
 id: "author_001",
 name: "山田太郎",
 nationality: "日本",
 },
},
```

```

 isbn: "978-4-XXXX-XXXX-X",
 category: "technology",
 pages: 400,
 publishedDate: "2025-06-01",
 description: "TypeScriptの基礎から実践まで学べる一冊",
 };

// 読書記録
const myReading: ReadingRecord = {
 id: "book_001",
 title: "TypeScript実践ガイド",
 author: {
 id: "author_001",
 name: "山田太郎",
 },
 isbn: "978-4-XXXX-XXXX-X",
 category: "technology",
 pages: 400,
 publishedDate: "2025-06-01",
 status: "reading",
 startDate: "2025-07-01",
 currentPage: 150,
 notes: "第5章まで読了。ジェネリクスの説明がわかりやすい。",
};

// フィルターを使った検索
const filter: BookFilter = {
 status: "reading",
 category: "technology",
 sortBy: "title",
 sortOrder: "asc",
};

// API レスポンスの型適用
const response: ApiResponse<ReadingRecord> = {
 data: myReading,
 success: true,
 message: "書籍情報を取得しました",
};

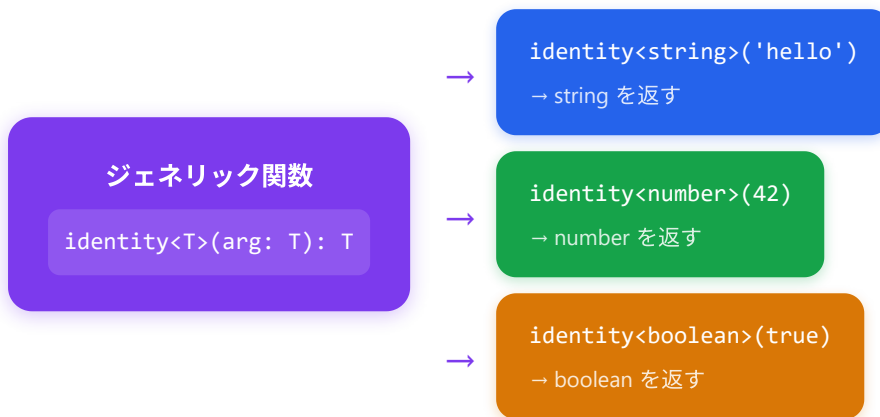
// ページネーション付きレスポンス
const listResponse: PaginatedResponse<ReadingRecord> = {
 data: [myReading],
 total: 1,
 page: 1,
 perPage: 10,
 totalPages: 1,
};

```

## 5. ジェネリクス

### 5.1 基本概念

ジェネリクス（Generics）は、型を「パラメータ化」する仕組みです。関数やクラス、インターフェースを定義する時点では型を決めず、使う時点で具体的な型を指定できます。



```
// --- ジェネリクスなしの場合（問題あり） ---
```

```
// 方法1: any を使う → 型安全性が失われる
```

```
function identityAny(arg: any): any { // any を使えば何でも受け
 return arg;
}
```

```
const result1 = identityAny("hello"); // result1 は any 型 (string 情報が失われる) // 結果も
```

```
// 方法2: 型ごとに関数を作る → コードの重複
```

```
function identityString(arg: string): string { // string 専用版。
 return arg;
}
```

```
function identityNumber(arg: number): number { // number 専用版。型が増
 return arg;
}
```

```
// =====
// ジェネリクスを使った場合 - 「型を変数化」して1つの関数で全てに対応
// =====

// `` の T は「Type」の頭文字。慣習的によく使われる。
// 「この関数は何かの型 T を受け取り、同じ T を返すよ」という宣言。
//
// arg: T → 引数の型は T
// : T → 戻り値の型も T (引数と同じ型)
function identity<T>(arg: T): T {
 return arg; // 受け取ったものをそのまま返す
}

// ----- 使用例(1): 型を明示的に指定する -----
// `` のように山カッコで型を渡すことを「型引数を渡す」と呼ぶ。
const str = identity<string>("hello");
// ↑ T = string と決まるので、引数は string 限定、戻り値も string
console.log(str);
// ▼ 実行結果
// hello
// ▼ str の型: string
// (number を渡そうとするとコンパイルエラー: identity<string>(42) は ✖)

const num = identity<number>(42);
console.log(num);
// ▼ 実行結果
// 42
// ▼ num の型: number

const bool = identity<boolean>(true);
console.log(bool);
// ▼ 実行結果
// true
// ▼ bool の型: boolean

// ----- 使用例(2): 型を省略する (型推論にお任せ) -----
// 引数の値 "world" を見て、TypeScript が「あ、T = string ですね」と自動推論する。
// 多くの場合、型引数を書かなくてOK。
const inferred = identity("world");
console.log(inferred);
// ▼ 実行結果
// world
// ▼ inferred の型: string (自動推論された)
```



ジェネリクスのありがたみ: 上の `identity` 関数は、もし型を毎回手書きだったら `identityString` `identityNumber` `identityBoolean` ... と無限に増えてしまう。<T> 1つで全てに対応できる。配列メソッドの `Array.prototype.map<U>(callback)` などはまさにこの仕組みで動いている。

```
// --- ジェネリクスの慣習的な型パラメータ名 ---
// T: Type (一般的な型)
// U: 2番目の型パラメータ
// K: Key (オブジェクトのキー)
// V: Value (オブジェクトの値)
// E: Element (要素)
// R: Return (戻り値)

// 複数の型パラメータ
function pair<T, U>(first: T, second: U): [T, U] { // <T, U> で2種類の型パラメータ
 return [first, second]; // 引数2つをタプルにして返す
}

const p1 = pair<string, number>("hello", 42); // [string, number] // 型を明示。T=string, U=number
const p2 = pair("田中", true); // [string, boolean] (型推論) // 引数から型を推論
```

## 5.2 実用的な例

### 例1: 配列操作のユーティリティ関数

```
// 配列の最初の要素を取得する関数
function getFirst<T>(arr: T[]): T | undefined { // <T> を導入。引数の
 return arr[0]; // [0] は1要素目。配
}

const firstFruit = getFirst(["りんご", "みかん"]); // string | undefined // T = string と
const firstNum = getFirst([10, 20, 30]); // number | undefined // T = number

// 配列の最後の要素を取得する関数
function getLast<T>(arr: T[]): T | undefined { // 同じく <T> を使う
 return arr.length > 0 ? arr[arr.length - 1] : undefined; // 三項演算子。長さが
}

// 配列をシャッフルする関数
function shuffle<T>(arr: T[]): T[] { // 配列を受け取り、
 const result = [...arr]; // ... はスプレッド
 for (let i = result.length - 1; i > 0; i--) { // 末尾から先頭手前
 const j = Math.floor(Math.random() * (i + 1)); // Math.random()
 [result[i], result[j]] = [result[j], result[i]]; // 分割代入による値
 }
 return result; // シャッフル後の配
}

const shuffledFruits = shuffle(["りんご", "みかん", "バナナ"]); // string[]
const shuffledNums = shuffle([1, 2, 3, 4, 5]); // number[]
```

## 例2: ジェネリックなインターフェース

```
// API レスポンスの汎用型
interface ApiResponse<T> {
 data: T;
 status: number;
 message: string;
 timestamp: Date;
}

// ユーザーデータ用のレスポンス
interface User {
 id: number;
 name: string;
 email: string;
}

const userResponse: ApiResponse<User> = {
 data: {
 id: 1,
 name: "田中太郎",
 email: "tanaka@example.com",
 },
 status: 200,
 message: "成功",
 timestamp: new Date(),
};

// 書籍データ用のレスポンス
interface Book {
 id: string;
 title: string;
}

const bookResponse: ApiResponse<Book[]> = {
 data: [
 { id: "1", title: "TypeScript入門" },
 { id: "2", title: "React実践ガイド" },
],
 status: 200,
 message: "成功",
 timestamp: new Date(),
};
```

### 例3: 制約付きジェネリクス (extends)

```
// T は必ず length プロパティを持つ型に制限
function getLength<T extends { length: number }>(arg: T): number { // <T extends X> は
 return arg.length; // length プロパティ
}

getLength("hello"); // OK: string は length を持つ → 5 // 文字列は length を
getLength([1, 2, 3]); // OK: 配列は length を持つ → 3 // 配列も length を持
getLength({ length: 10 }); // OK: length プロパティがある → 10 // 自前で { length: 10

// getLength(42);
// エラー: Argument of type 'number' is not assignable to
// parameter of type '{ length: number; }'
// → number には length プロパティがない // number に length は

// オブジェクトのキーに制限をかける
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] { // K は「T のキーの中
 return obj[key];
}

const user = { name: "田中", age: 30, email: "tanaka@example.com" }; // ↑ user の型は { n

const name = getProperty(user, "name"); // string // k="name" なので戻
const age = getProperty(user, "age"); // number // k="age" なので no

// getProperty(user, "address"); // "address" は use
// エラー: Argument of type '"address"' is not assignable to
// parameter of type '"name" | "age" | "email"'
```

#### 例4: ジェネリックなクラス

```
// スタック (Last In, First Out) のデータ構造
class Stack<T> {
 private items: T[] = [];

 push(item: T): void {
 this.items.push(item);
 }

 pop(): T | undefined {
 return this.items.pop();
 }

 peek(): T | undefined {
 return this.items[this.items.length - 1];
 }

 isEmpty(): boolean {
 return this.items.length === 0;
 }

 size(): number {
 return this.items.length;
 }
}

// 数値スタック
const numberStack = new Stack<number>();
numberStack.push(10);
numberStack.push(20);
numberStack.push(30);
console.log(numberStack.pop()); // 30
console.log(numberStack.peek()); // 20

// 文字列スタック
const stringStack = new Stack<string>();
stringStack.push("TypeScript");
stringStack.push("React");
console.log(stringStack.pop()); // "React"
```

## 6. ユニオン型とリテラル型

---

### 6.1 ユニオン型

ユニオン型（Union Type）は、複数の型のいずれかを受け入れる型です。|（パイプ）で型を区切ります。

```
// =====
// ユニオン型の基本 - 「A型 または B型」を受け入れる型
// =====

// (1) string 型 または number 型に入れられる変数を宣言する
// let は const と違って後から再代入できる。
// | は「または」の意味 (AND ではなく OR)。
let value: string | number;

value = "hello"; // OK (string なので許可)
value = 42; // OK (number なので許可)
// value = true;
// ▼ エラー
// Type 'boolean' is not assignable to type 'string | number'.
// (boolean は string|number に代入不可)

// =====
// 関数の引数にユニオン型を使う場合の「型の絞り込み」
// =====
// ユニオン型のままでは「両方の型に共通するメソッドしか」呼べない。
// typeof 演算子などで「いま中身が何の型か」を確かめてから使う。
// この技法を「型ガード (Type Guard)」と呼ぶ。

function formatId(id: string | number): string {
 // (1) typeof 演算子は「値の型を文字列で返す」演算子
 // 文字列なら "string"、数値なら "number" を返す。
 if (typeof id === "string") {
 // (2) この if ブロックの中だけ、TS は id を string として扱ってくれる。
 // なので string 専用メソッド .toUpperCase() が使える。
 return id.toUpperCase(); // 例: "abc" → "ABC"
 } else {
 // (3) else 側では「string ではない」=「number しか残らない」と推論される。
 // .toString() は数値を文字列に変換するメソッド。
 // .padStart(5, "0") は「5文字に満たないなら左を 0 で埋める」メソッド。
 return id.toString().padStart(5, "0"); // 例: 42 → "00042"
 }
}

console.log(formatId("abc"));
// ▼ 実行結果
// ABC

console.log(formatId(42));
// ▼ 実行結果
// 00042

// =====
```

```
// 配列のユニオン型 - 配列の中に複数の型が混ざってもOKにする
// =====
// 注意: () で囲まないと意味が変わる!
// string | number[] → 「string 1個」または「number の配列」
// (string | number)[] → 「string と number が混在した配列」 (こちらが正しい)
const mixed: (string | number)[] = [1, "two", 3, "four"];
console.log(mixed);
// ▼ 実行結果
// [1, 'two', 3, 'four']
```

## 6.2 リテラル型

リテラル型 (Literal Type) は、特定の値のみを許可する型です。



```
// =====
// 文字列リテラル型 - 「決められた文字列のどれか」だけを許す型
// =====
// type で名前を付け、許可する値を | で並べる。Enum の代わりによく使う。

type Direction = "north" | "south" | "east" | "west";
// ↑ この4つの文字列以外は受け付けない

let dir: Direction = "north"; // OK (許可されている値)
// dir = "up";
// ▼ エラー
// Type '"up"' is not assignable to type 'Direction'.

console.log(dir);
// ▼ 実行結果
// north

// =====
// 数値リテラル型 - 「決められた数値のどれか」だけを許す型
// =====
// サイコロの目 (1~6) のように値が固定の場面で使う。
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;

let roll: DiceRoll = 3; // OK
// roll = 7;
// ▼ エラー
// Type '7' is not assignable to type 'DiceRoll'.

// =====
// 真偽値リテラル型 - true だけ、または false だけを許す
// =====
// 滅多に使わないが、特定の値しか取らないフラグを表現したいとき便利。
type True = true;
let flag: True = true; // OK
// flag = false; // ▼ エラー: Type 'false' is not assignable to type 'true'.
```

## 6.3 書籍管理アプリでの実践例

```
// ===== 読書ステータスの定義 =====

type ReadingStatus = "want-to-read" | "reading" | "completed"; // 3つの文字列リテラル

interface Book { // 書籍の基本型。
 id: string;
 title: string;
 author: string;
 status: ReadingStatus; // ステータスは ReadingStatus
}

// --- 正しい使い方 ---
const book1: Book = {
 id: "1",
 title: "TypeScript入門",
 author: "山田太郎",
 status: "reading", // 許可された値。
};

const book2: Book = {
 id: "2",
 title: "React実践ガイド",
 author: "佐藤花子",
 status: "completed",
};

const book3: Book = {
 id: "3",
 title: "Next.js入門",
 author: "鈴木一郎",
 status: "want-to-read",
};

// --- エラーになる例 ---
const invalidBook: Book = {
 id: "4",
 title: "間違った書籍",
 author: "テスト太郎",
 status: "finished", // エラー! // "finished" は ReadingStatus
 // Type '"finished"' is not assignable to type 'ReadingStatus'
 // → "want-to-read" | "reading" | "completed" のどれかにしてください
};
```

```

// ===== ステータスに応じた処理 =====

function getStatusLabel(status: ReadingStatus): string {
 switch (status) {
 case "want-to-read":
 return "読みたい";
 case "reading":
 return "読書中";
 case "completed":
 return "読了";
 }
 // すべてのケースを処理しているため、default 不要
 // TypeScript がすべてのケースが網羅されていることを保証
}

function getStatusColor(status: ReadingStatus): string {
 switch (status) {
 case "want-to-read":
 return "#f39c12"; // オレンジ
 case "reading":
 return "#3498db"; // 青
 case "completed":
 return "#27ae60"; // 緑
 }
}

function getStatusIcon(status: ReadingStatus): string {
 const icons: Record<ReadingStatus, string> = {
 "want-to-read": "bookmark",
 reading: "book-open",
 completed: "check-circle",
 };
 return icons[status];
}

// ===== フィルタリング =====

function filterBooksByStatus(
 books: Book[],
 status: ReadingStatus
): Book[] {
 return books.filter((book) => book.status === status);
}

const allBooks: Book[] = [book1, book2, book3];
const readingBooks = filterBooksByStatus(allBooks, "reading");
const completedBooks = filterBooksByStatus(allBooks, "completed");

// ===== ステータスの変更 =====

```

```

function updateBookStatus(// book に新しいステータスを渡す
 book: Book,
 newStatus: ReadingStatus
): Book {
 return { ...book, status: newStatus }; // ... はスプレッド演算子
}

const updatedBook = updateBookStatus(book1, "completed"); // book1 の status を completed に更新
console.log(updatedBook.status); // "completed"

// 無効なステータスへの変更はコンパイルエラー
// updateBookStatus(book1, "abandoned"); // ReadingStatus 型ではない
// エラー: Argument of type '"abandoned"' is not assignable to type 'ReadingStatus'

```

```

// ===== 判別可能なユニオン型 (Discriminated Union) =====
// 判別可能なユニオン型 = 「共通のキー (タグ)」を使って、ユニオン型の中の
// どのバリエーションかを区別できる型。TypeScript で複雑な状態を扱う鉄板パターン。

// ステータスに応じて異なる追加情報を持つ型
type BookWithDetails =
 | { // ユニオンの1つ目: status: "want-to-read" // ↑ 「判別タグ」
 status: "want-to-read";
 title: string;
 reason: string; // 読みたい理由 // この種類だけ追加情報を持つ
 }
 | { // 2つ目: status: "reading"
 status: "reading";
 title: string;
 currentPage: number; // 読書中だけ追加情報を持つ
 totalPages: number;
 }
 | { // 3つ目: status: "completed"
 status: "completed";
 title: string;
 rating: 1 | 2 | 3 | 4 | 5; // 数値リテラル
 review?: string; // review は任意
 };

function displayBookInfo(book: BookWithDetails): string {
 switch (book.status) {
 case "want-to-read": // book.status: "want-to-read"
 // ここでは book.reason にアクセス可能 // この case 内では book.reason が必ず存在
 return `「${book.title}」を読みたい (理由: ${book.reason})`;

 case "reading":
 // ここでは book.currentPage, book.totalPages にアクセス可能 // この case では book.currentPage, book.totalPages が必ず存在
 const progress = Math.round(// Math.round() は整数を返す
 (book.currentPage / book.totalPages) * 100 // 進捗率 =
);
 return `「${book.title}」を読書中 (進捗: ${progress}%)`;

 case "completed":
 // ここでは book.rating, book.review にアクセス可能 // この case では book.rating, book.review が必ず存在
 const stars = "★".repeat(book.rating) + "☆".repeat(5 - book.rating); // .repeat() は文字列を指定した回数だけ繰り返す
 return `「${book.title}」読了 ${stars}`;
 }
}

// 使用例
console.log(
 displayBookInfo({
 status: "reading",
 title: "TypeScript入門",
 })
);

```

```

 currentPage: 150,
 totalPages: 400,
 })
};
// => 「TypeScript入門」を読書中（進捗: 38%）

console.log(
 displayBookInfo({
 status: "completed",
 title: "React実践ガイド",
 rating: 4,
 review: "とても実践的でした",
 })
);
// => 「React実践ガイド」読了 ★★★★★

```

## 7. TypeScript の設定（tsconfig.json）

### 7.1 tsconfig.json とは

**tsconfig.json** は TypeScript プロジェクトの設定ファイルです。コンパイラのオプション、対象ファイル、出力先などを指定します。

```

{
 "compilerOptions": {
 "target": "ES2022", // TypeScript コンパイラ（tsc）に渡す設定。
 "module": "ESNext", // コンパイル後の JavaScript のバージョン。新しい
 "lib": ["ES2022", "DOM", "DOM.Iterable"], // モジュールシステム。"ESNext" = ES Modules (モジュール)
 "strict": true, // 使える組み込み型の集合。"DOM" を入れるとブラウ
 "esModuleInterop": true, // 厳格モード一括有効化。これだけで厳しい型チェッ
 "skipLibCheck": true, // CommonJS と ES Modules の互換性を良くする。
 "forceConsistentCasingInFileNames": true, // node_modules 内の型定義のチェックを省略してビ
 "resolveJsonModule": true, // ファイル名の大文字小文字を厳格にチェック。macOS
 "isolatedModules": true, // import data from "./foo.json" のように JSO
 "noEmit": true, // 1ファイルだけ見て変換できる前提を強制。Babel や
 "jsx": "react-jsx", // tsc から .js ファイルを出力しない。Next.js
 "baseUrl": ".", // JSX (React の <Component/> 構文) の変換方式
 "paths": { // パスエイリアスの基準ディレクトリ。"." はプロシ
 " "@/*": [".src/*"] // 短縮パスの定義。
 }, // "@foo" と書けば ".src/foo" を指す。深いネ
 },
 "include": ["src/**/*"], // コンパイル対象。** は「任意の階層」、* は「任意
 "exclude": ["node_modules", "dist"] // 除外対象。ライブラリやビルド成果物はチェックしな
}

```

## 7.2 主要な設定項目

設定項目	説明	推奨値	備考
<code>target</code>	コンパイル先の JavaScript バージョン	<code>"ES2022"</code>	モダンブラウザ対象なら ES2022 以降
<code>module</code>	モジュールシステム	<code>"ESNext"</code>	ES Modules を使用
<code>lib</code>	使用する型定義ライブラリ	<code>["ES2022", "DOM"]</code>	ブラウザ用に DOM を含める
<code>strict</code>	すべての厳格チェックを有効化	<code>true</code>	必ず <code>true</code> にする
<code>noEmit</code>	JavaScript ファイルを出力しない	<code>true</code>	Next.js 等のバンドラーを使う場合
<code>jsx</code>	JSX の処理方法	<code>"react-jsx"</code>	React 17+ の新しい JSX Transform
<code>esModuleInterop</code>	CommonJS/ESModule の相互運用	<code>true</code>	import 構文の互換性向上
<code>skipLibCheck</code>	型定義ファイルのチェックを省略	<code>true</code>	コンパイル速度の向上
<code>forceConsistentCasingInFileNames</code>	ファイル名の大文字小文字を区別	<code>true</code>	OS 間の差異を防止
<code>resolveJsonModule</code>	JSON ファイルの import を許可	<code>true</code>	JSON をモジュールとして読み込める
<code>isolatedModules</code>	ファイル単位のトランスパイルを保証	<code>true</code>	Babel 等との互換性
<code>baseUrl</code>	モジュール解決のベースパス	<code>"."</code>	パスエイリアスの基準
<code>paths</code>	パスエイリアスの定義	<code>{"@/*": ["./src/*"]}</code>	<code>@/components</code> のような短縮パス

### strict モードに含まれる個別オプション

`"strict": true` を設定すると、以下のオプションがすべて有効になります。

オプション	説明
<code>strictNullChecks</code>	<code>null</code> / <code>undefined</code> を他の型と区別する
<code>strictFunctionTypes</code>	関数の引数の型チェックを厳密にする
<code>strictBindCallApply</code>	<code>bind</code> , <code>call</code> , <code>apply</code> の引数を厳密にチェック
<code>strictPropertyInitialization</code>	クラスプロパティの初期化を必須にする
<code>noImplicitAny</code>	暗黙の <code>any</code> 型を禁止する
<code>noImplicitThis</code>	暗黙の <code>this</code> 型を禁止する
<code>alwaysStrict</code>	JavaScript の strict mode を有効にする
<code>useUnknownInCatchVariables</code>	catch の変数を <code>unknown</code> 型にする

## 7.3 Next.js での TypeScript 設定

Next.js プロジェクトでは、`next.config.ts` と `tsconfig.json` の両方で TypeScript の設定を行います。Next.js が自動的に最適な `tsconfig.json` を生成してくれます。



```
// Next.js プロジェクトの tsconfig.json (自動生成される)
{
 "compilerOptions": {
 "target": "ES2017", // 古めのブラウザも考慮した出力バージョン。
 "lib": ["dom", "dom.iterable", "esnext"], // ブラウザの DOM 系 + 最新の JS 機能の型を有
 "allowJs": true, // .js ファイルも TypeScript と同居できるよ
 "skipLibCheck": true,
 "strict": true, // 厳格モード ON。
 "noEmit": true, // Next.js 側のバンドラ (SWC) が出力するの
 "esModuleInterop": true,
 "module": "esnext",
 "moduleResolution": "bundler", // バンドラ向けの解決方式。最新の Next.js 推
 "resolveJsonModule": true,
 "isolatedModules": true,
 "jsx": "preserve", // JSX を変換せずそのまま残す。あとで Next.js
 "incremental": true, // 前回の型情報を再利用してビルドを高速化。
 "plugins": [
 {
 "name": "next" // Next.js 用 TypeScript プラグインを有効
 }
],
 "paths": {
 "@/*": [".src/*"] // @/foo → ./src/foo のショートカット。
 }
 },
 "include": [
 "next-env.d.ts", // Next.js が自動生成する型定義ファイル。
 "**/*.ts", // すべての .ts ファイル。
 "**/*.tsx", // JSX を含む .tsx ファイル。
 ".next/types/**/*.ts" // .next ビルド出力中の型ファイル。
],
 "exclude": ["node_modules"]
}
```

ポイント: Next.js では `"jsx": "preserve"` が使われます。これは JSX の変換を Next.js (SWC) に任せるためです。 `"noEmit": true` なので TypeScript コンパイラ自身は JavaScript を出力しません。

```
// Next.js 特有の型の例

// ページコンポーネントの型
import type { Metadata } from "next"; // import type は「型情報だけ」

export const metadata: Metadata = { // ページのメタ情報（ブラウザの表示に反映）
 title: "書籍管理アプリ",
 description: "あなたの読書を管理するアプリです",
};

export default function HomePage() { // export default = このファイルのデフォルトエクスポート
 return <h1>書籍管理アプリ</h1>; // JSX。HTML のように見えるが、実際は JavaScript のコード
}

// Server Component のデータ取得
interface PageProps { // Next.js のページコンポーネントの型
 params: Promise<{ id: string }>; // URL のパスパラメータ。Next.js のドキュメントを参照
 searchParams: Promise<{ [key: string]: string | string[] | undefined }>; // URL のクエリパラメータ
}

export default async function BookPage({ params }: PageProps) { // async = 非同期関数。{ params } は PageProps の型
 const { id } = await params; // await は Promise の中で値を待つ
 // サーバーサイドでデータを取得
 const book = await fetchBook(id); // fetchBook(id) も Promise
 return <div>{book.title}</div>; // JSX の中で { 式 } と書く
}
```

## 8. よくあるエラーと対処法

TypeScript を使い始めると、必ず遭遇するエラーがあります。ここでは代表的なエラーとその解決方法を詳しく解説します。

### 8.1 Type 'X' is not assignable to type 'Y'

最も頻出するエラーです。ある型の値を、互換性のない別の型に代入しようとした際に発生します。

// ===== エラーパターン1: プリミティブ型の不一致 =====

// 悪い例

```
const age: number = "25"; // number 型に "25" (文字列) を
// エラー: Type 'string' is not assignable to type 'number'
```

// 良い例

```
const age: number = 25; // 数値リテラルを直接入れる。
// または、文字列から変換する場合
const age: number = parseInt("25", 10); // parseInt は文字列を整数に変換
const age: number = Number("25"); // Number() は文字列・boolean
```

// ===== エラーパターン2: オブジェクト型の不一致 =====

```
interface User { // 型を定義。
 name: string;
 age: number;
}
```

// 悪い例

```
const user: User = {
 name: "田中",
 age: "30", // string を number に代入 // age は number でなければなら
};
// エラー: Type 'string' is not assignable to type 'number'
```

// 良い例

```
const user: User = {
 name: "田中",
 age: 30, // 数値に修正。
};
```

// ===== エラーパターン3: ユニオン型のリテラルが一致しない =====

```
type Status = "active" | "inactive"; // 許可される値は "active" か
```

// 悪い例

```
const status: Status = "enabled"; // "enabled" は含まれていない。
// エラー: Type '"enabled"' is not assignable to type 'Status'
```

// 良い例

```
const status: Status = "active";
```

```
// ===== エラーパターン4: 変数の型が広すぎる =====

type Color = "red" | "blue" | "green";

// 悪い例
let colorName = "red"; // string と推論される // let は再代入されうるので「広い」
const color: Color = colorName; // string は Color (3リテラル) ではない
// エラー: Type 'string' is not assignable to type 'Color'

// 良い例 (方法1: as const を使う)
const colorName = "red" as const; // "red" リテラル型と推論 // as const は「値をできる限り狭くする」
const color: Color = colorName; // OK

// 良い例 (方法2: 型注釈を使う)
const colorName: Color = "red"; // 最初から Color 型として宣言

// 良い例 (方法3: satisfies を使う)
const colorName = "red" satisfies Color; // "red" リテラル型かつ Color として検証 // satisfies
```

## 8.2 Object is possibly 'undefined'

**strictNullChecks** が有効な場合に発生する、`null` や `undefined` の可能性がある値にアクセスしようとしたときのエラーです。

```
// ===== エラーパターン1: 配列の要素アクセス =====

const fruits = ["りんご", "みかん", "バナナ"];

// 悪い例
const first: string = fruits[0]; // noUncheckedIndexedAccess
// エラー (strict 設定次第): Type 'string | undefined' is not assignable to type 'string'
// 配列のインデックスアクセスは undefined を返す可能性がある

// 良い例 (方法1: undefined チェック)
const first = fruits[0]; // 型注釈を省くと string | undefined
if (first !== undefined) { // undefined でないか確認
 console.log(first.toUpperCase()); // OK // ガード内では string になる
}

// 良い例 (方法2: デフォルト値)
const first = fruits[0] ?? "デフォルト"; // ?? で undefined の場合のデフォルト値
console.log(first.toUpperCase()); // OK
```

```
// ===== エラーパターン2: オプショナルプロパティ =====

interface User {
 name: string;
 email?: string; // オプショナル (string | undefined) // ? を付けると省略可能なプロ
}

const user: User = { name: "田中" }; // email を省略。値は unde

// 悪い例
console.log(user.email.toUpperCase()); // user.email は string|
// エラー: Object is possibly 'undefined'
// email は設定されていないかもしれない

// 良い例 (方法1: if チェック)
if (user.email) { // truthy チェック (unde
 console.log(user.email.toUpperCase()); // OK // ガード内では string は
}

// 良い例 (方法2: オプショナルチェイニング)
console.log(user.email?.toUpperCase()); // undefined なら undefined を返す // ?. を付けると

// 良い例 (方法3: null 合体演算子と組み合わせ)
console.log(user.email?.toUpperCase() ?? "メール未設定"); // 結果が undefined なら ?
```

```
// ===== エラーパターン3: Map.get() の戻り値 =====

const userMap = new Map<string, string>(); // Map は「キーと値のペアを
userMap.set("user1", "田中"); // .set(キー, 値) で追加。

// 悪い例
const name: string = userMap.get("user1"); // .get(キー) は「キーが存
// エラー: Type 'string | undefined' is not assignable to type 'string'
// Map.get() は undefined を返す可能性がある

// 良い例
const name = userMap.get("user1"); // 型は string | undefin
if (name !== undefined) {
 console.log(name); // OK
}

// または
const name = userMap.get("user1") ?? "不明"; // 未設定キーに対するデフォ
```

```
// ===== エラーパターン4: document.querySelector =====

// 悪い例
const button = document.querySelector("#submit-btn"); // querySelector は CSS で指定された要素を返す
button.addEventListener("click", handleClick); // null かもしれないものに
// エラー: Object is possibly 'null'
// querySelector は要素が見つからない場合 null を返す

// 良い例 (方法1: null チェック)
const button = document.querySelector("#submit-btn");
if (button) { // null/undefined でない
 button.addEventListener("click", handleClick); // ガード内では非 null
}

// 良い例 (方法2: 存在が確実な場合は Non-null assertion)
const button = document.querySelector("#submit-btn")!; // 末尾の ! は「非nullアサーション」
// ただし、要素が存在しない場合は実行時エラーになるので注意
button.addEventListener("click", handleClick);

// 良い例 (方法3: 型を絞り込む)
const button = document.querySelector<HTMLButtonElement>("#submit-btn"); // <HTMLButtonElement>
if (button instanceof HTMLButtonElement) { // instanceof は「特定の型かどうか」を確認
 button.disabled = true; // HTMLButtonElement のプロパティにアクセス可能 // ガード内では HTMLButtonElement
}
```

### 8.3 Property does not exist on type

オブジェクトに存在しないプロパティにアクセスしようとした際に発生するエラーです。

```
// ===== エラーパターン1: タイプミス =====

interface User {
 name: string;
 email: string;
}

const user: User = { name: "田中", email: "tanaka@example.com" };

// 悪い例
console.log(user.emial); // emial (タイプミス)。User に emial というプロパティは存在しない
// エラー: Property 'emial' does not exist on type 'User'.
// Did you mean 'email'? // TypeScript が「もしかして email だったの？」と提案

// 良い例
console.log(user.email);
```

```
// ===== エラーパターン2: 型定義にプロパティが不足 =====

interface Product {
 name: string;
 price: number;
}

const product: Product = { name: "ペン", price: 150 };

// 悪い例
console.log(product.description); // Product 型に description
// エラー: Property 'description' does not exist on type 'Product'

// 良い例 (方法1: 型定義を修正)
interface Product {
 name: string;
 price: number;
 description?: string; // プロパティを追加 // オptional (?) で追加する
}

// 良い例 (方法2: オブジェクトリテラルにプロパティを追加する場合)
const productWithDesc = { ...product, description: "赤いボールペン" }; // スプレッドで既存プロ
```

```
// ===== エラーパターン3: ユニオン型での共通でないプロパティ =====
```

```
interface Dog {
 kind: "dog";
 bark(): void;
}
```

// 判別タグ。"dog" リテラル型  
// bark メソッド (戻り値なし)

```
interface Cat {
 kind: "cat";
 meow(): void;
}
```

```
type Animal = Dog | Cat; // Dog または Cat。
```

```
// 悪い例
```

```
function makeSound(animal: Animal) {
 animal.bark();
 // エラー: Property 'bark' does not exist on type 'Animal'
 // Property 'bark' does not exist on type 'Cat'
 // → Animal が Cat の場合、bark() は存在しない
}
```

// Cat に bark は無い。ユニオン型では、両方の型に存在するプロパティでしか参照できない

```
// 良い例 (型の絞り込み)
```

```
function makeSound(animal: Animal) {
 if (animal.kind === "dog") {
 animal.bark(); // OK: Dog 型として認識
 } else {
 animal.meow(); // OK: Cat 型として認識
 }
}
```

// kind を使ってどちらか判定  
// ガード内では Dog に絞り込める  
// else 側では Cat に絞り込める



```
// ===== エラーパターン4: API レスポンスの型が不足 =====

// 悪い例: JSON.parse の結果は any ではなく unknown として扱うべき
async function fetchData() {
 const response = await fetch("/api/data");
 const data = await response.json(); // any 型

 // この時点では data の構造が不明
 console.log(data.items.length); // 何のチェックも無しに data
 // ← 実行時エラーの可能性あり
}

// 良い例: 型を明示的に定義
interface ApiData {
 items: string[];
 total: number;
}

// 返ってくる JSON の構造を

async function fetchData(): Promise<ApiData> {
 const response = await fetch("/api/data");
 const data: ApiData = await response.json();

 // 戻り値の型を Promise<ApiData>
 // 取得結果を ApiData 型と

 // 型安全にアクセスできる
 console.log(data.items.length); // items は string[] と分
 console.log(data.total);

 return data;
}
```

## 8.4 エラー対処のまとめフローチャート



## まとめ

この章では、TypeScript の基礎として以下の内容を学びました。

トピック	学んだこと
TypeScript とは	JavaScript のスーパーセットであり、型安全性を提供する
基本的な型	<code>string</code> , <code>number</code> , <code>boolean</code> , <code>array</code> , <code>tuple</code> , <code>object</code> , <code>any</code> , <code>unknown</code> , <code>never</code> , <code>void</code> , <code>null</code> , <code>undefined</code>
型注釈と型推論	明示的に型を書く方法と、TypeScript が自動で型を判断する仕組み
interface と type	オブジェクトの形を定義する2つの方法とその使い分け
ジェネリクス	型をパラメータ化して再利用性を高める仕組み
ユニオン型とリテラル型	複数の型や特定の値のみを許可する型定義
tsconfig.json	TypeScript プロジェクトの設定ファイル
よくあるエラー	代表的な型エラーの原因と対処法

## 次の章へ

次の章では、**React の基礎**を学び、TypeScript と React を組み合わせた開発方法を習得していきます。この章で学んだ型の知識は、React コンポーネントの Props や State の定義で活用されます。